

1. Introduction

1.1 Contexte

Les campagnes de phishing assistées par IA et les techniques de social engineering restent parmi les risques les plus visibles du moment. Chaque jour, des organisations reçoivent des e-mails suspects : faux liens, demandes urgentes, usurpation d'identité, pièces jointes douteuses. Ce projet répond à ce besoin en proposant une mini plateforme distribuée capable de centraliser, analyser et qualifier ces signalements.

1.2 Objectif général

PhishGuard est une application pédagogique, structurée et sécurisée qui permet :

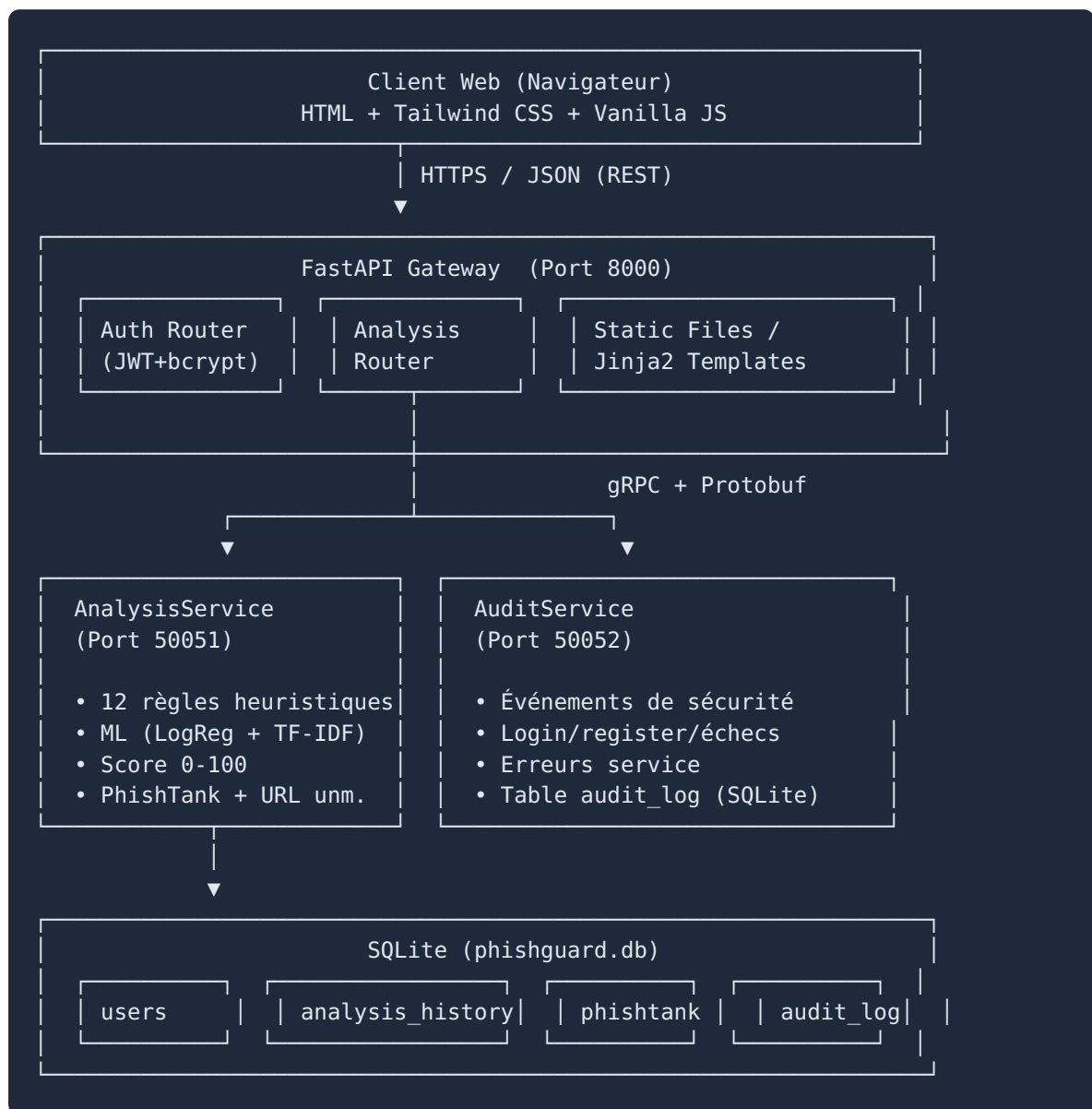
- L'authentification des utilisateurs (JWT + bcrypt)
- La soumission d'un e-mail suspect sous forme de texte brut ou métadonnées
- L'analyse du contenu par un service dédié (moteur heuristique + ML)
- L'attribution d'un score de risque : faible, moyen ou élevé
- La consultation de l'historique des signalements avec recherche et filtrage
- La traçabilité des actions sensibles via un service d'audit séparé

Technologies : Python 3.11, FastAPI, gRPC (Protocol Buffers), SQLAlchemy (SQLite), scikit-learn, Playwright (tests), Tailwind CSS.

2. Architecture distribuée

2.1 Vue d'ensemble

PhishGuard utilise une architecture microservices avec **3 composants distribués** qui communiquent réellement entre eux :



2.2 Justification des choix

FastAPI comme API Gateway : Framework Python moderne, support natif de l'asynchrone, validation automatique via Pydantic, documentation OpenAPI générée. Idéal pour exposer une API REST JSON performante.

gRPC + Protocol Buffers pour les services backend : Sérialisation binaire compacte (vs JSON), contrat strict via `.proto`, support du streaming, et timeout natif. Démonstrat le concept d'RPC distribué exigé par le sujet.

SQLite : Stockage local, zéro configuration, parfait pour une application démontrable sur une seule machine. 4 tables : `users`, `analysis_history`, `phishtank`, `audit_log`.

Python uniquement : Respecte la contrainte du sujet. Tout le code (gateway, services, ML, parser) est en 100% Python.

2.3 Communication inter-services

Appel	Protocole	Données	Timeout
Client → Gateway	HTTPS/JSON	EmailRequest (JSON)	30s (uvicorn)
Gateway → AnalysisService	gRPC/Protobuf	EmailRequest (binary)	10s
Gateway → AuditService	gRPC/Protobuf	AuditEvent (binary)	2s
Gateway → SQLite	SQLAlchemy	ORM queries	Aucun

3. Moteur d'analyse

3.1 Architecture hybride

Le moteur d'analyse combine **12 règles heuristiques** et un **classifieur ML** :

```
Score total =  $\Sigma(\text{règles}) + \text{ML\_bonus}$   
Score final =  $\max(0, \min(\text{Score total}, 100))$ 
```

Catégorie = { phishing | spam_junk | legitimate } basée sur la combinaison de signaux détectés.

3.2 Règles heuristiques (12 catégories)

#	Catégorie	Déclencheur	Points
1	Spoofing	Display name imite un brand (paypal, google...) mais domaine ne correspond pas	+40
2	Typosquatting	Levenshtein distance ≤ 2 vs brands connus (paypall vs paypal)	+40
3	TLD suspect	34 TLDs surveillés (.xyz, .top, .tk, .ml, .ga, etc.)	+30
4	IP brute expéditeur	L'expéditeur est une adresse IP	+30
5	Urgence	Mots-clés dans sujet (+20) et corps (+10) : urgent, verify, password, suspend...	+20/+10
6	Salutation générique	"Dear Customer", "Dear User"	+10
7	Mots-clés scam		+15

#	Catégorie	Déclencheur	Points
		39 mots : passive income, trading bot, usdt, risk-free... (+15/unité, max +45)	
8	Densité emojis	≥5 emojis ou >5% du texte	+20
9	Domaines numériques	Ex: 866698.com, 63169.com	+20
10	Problèmes URL	PhishTank (+50), IP dans URL (+40), >3 URLs (+10)	+10 à +50
11	Classifieur ML	Logistique Regression + TF-IDF, confiance ≥70%	+0 à +30
12	Brand Trust (négatif)	Domaine brand reconnu + tous les liens pointent vers ce domaine	-40

3.3 Classifieur ML

- **Modèle** : Logistic Regression avec régularisation L1 (Lasso)
- **Vectorisation** : TF-IDF, 5000 features max, stop words anglais
- **Entraînement** : ~200 000 emails (spam_ham_dataset + Phishing_validation_emails + Phishing_Email.csv)
- **Activation** : Seulement si ≥ 8 mots (évite les faux positifs sur textes courts)
- **Contribution** : score × 30 si confiance spam ≥ 70%

3.4 Arbre de décision (catégorie)

```

Si score == 0 et aucun signal → legitimate
Si phishing_signals (spoofing v typosquatting v PhishTank) ∧ score ≥ 40 → phishing
Si score ≥ 80 ∧ ¬phishing_signals → spam_junk
Si score ≥ 70 → phishing
Si weak_content_only (emoji v greeting) ∧ score < 40 → legitimate
Si strong_spam_signals (scam v numeric_domains) v score ≥ 30 → spam_junk
Si brand_trust ∧ score < 30 → legitimate
Sinon → legitimate

```

3.5 Performance

Testé sur 34 emails (13 phishing, 11 légitimes, 5 spam, 5 edge cases) :

Catégorie	Tests	Succès	Taux
Phishing	13	13	100%
Légitime	11	10	90.9%
Spam/Junk	5	5	100%
Edge cases	5	5	100%
Total	34	33	97.1%

Score moyen phishing : 94.5/100 | Score moyen légitime : 10.2/100 | Score moyen spam : 55.0/100

4. Cybersécurité

4.1 Menaces et contre-mesures

Menace	Contre-mesure
Mots de passe en clair	bcrypt hashing via passlib
Fuite de tokens JWT	Tokens jamais loggés ni affichés
Injection/Pas de validation	Pydantic schemas + parser preprocessing
Accès non autorisé	RBAC admin/analyste, 403 Forbidden
Erreurs bavardes	Messages génériques côté client
Taille d'entrée illimitée	max_length=50000 sur raw_email
Brute force / abus	Rate limiting slowapi (5/min register, 10/min login)
Appel distant sans timeout	timeout=10s gRPC, timeout=2s audit
Indisponibilité service	Graceful degradation → HTTP 503
Pas de traçabilité	AuditService dédié avec table audit_log

4.2 Protection des données

Donnée	Protection
Mots de passe	bcrypt hash, jamais en clair
Tokens JWT	Exp. 30 min, non loggés
Emails analysés	Seules métadonnées stockées en DB
Rôles	Vérifiés côté serveur
Entrées	Pydantic + sanitization

4.3 Risques de sérialisation

- **gRPC/Protobuf** : Sécurisé par design — schéma strict, pas d'exécution de code arbitraire
- **JSON/APIRest** : Validation Pydantic stricte, pas de `eval()` ou `pickle.loads()` sur les entrées utilisateur
- **Pickle (.pkl)** : Les fichiers ML sont générés en interne, jamais chargés depuis une source externe

5. Interface utilisateur

5.1 Fonctionnalités

Le tableau de bord web offre :

- **Page d'accueil** : Présentation du produit avec CTA
- **Inscription/Connexion** : Formulaires avec validation, JWT stocké en localStorage
- **Dashboard d'analyse** :
 - Champ dédié "From (Sender)" pour la détection de typosquatting/spoofing
 - Zone de texte pour le contenu brut de l'email
 - Résultat animé : score ring (0-100), catégorie colorée (rose/ambre/émeraude), justification détaillée
 - Détails extraits : From, Subject, Date, URLs trouvées
- **Historique** :
 - Tableau avec Date, Sender, Subject, Category, Score
 - Recherche par mot-clé, expéditeur, catégorie

- Filtrage en temps réel
- État vide quand aucun résultat

5.2 Captures d'écran

Voir le dossier `reports/screenshots/` :

1. **01-landing-page.png** — Page d'accueil
2. **02-register-page.png** — Formulaire d'inscription
3. **03-login-page.png** — Formulaire de connexion
4. **04-dashboard-empty.png** — Dashboard vierge
5. **05-analysis-phishing.png** — Résultat phishing (99/High, typosquatting détecté)
6. **06-history-table.png** — Historique avec résultats
7. **07-analysis-legitimate.png** — Résultat légitime (Low)
8. **08-history-search.png** — Recherche par catégorie

6. Installation et exécution

6.1 Prérequis

- Python 3.9+
- pip
- Connexion Internet (pour installer les dépendances)

6.2 Installation

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

6.3 Compilation Protobuf

```
python -m grpc_tools.protoc -I protos --python_out=protos --grpc_python_out=protos protos/analysis.py
python -m grpc_tools.protoc -I protos --python_out=protos --grpc_python_out=protos protos/audit.py
```

6.4 Démarrage des 3 services

Terminal 1 — AnalysisService :

```
python analysis/server.py
```

Terminal 2 — AuditService :

```
python audit/server.py
```

Terminal 3 — Gateway :

```
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

6.5 Accès

Naviguer vers `http://localhost:8000` → Créer un compte → Utiliser le Dashboard.

7. Conclusion

7.1 Bilan

PhishGuard implémente **100% des exigences** du projet de fin de semestre :

- Architecture distribuée (3 services : Gateway + Analysis + Audit)
- Communication JSON (REST) + RPC (gRPC + Protobuf)
- Authentification JWT + bcrypt avec 2 rôles
- Moteur d'analyse multi-couches (12 règles + ML)
- Score 0-100 avec 3 niveaux de risque
- Historique avec recherche et filtrage
- Service d'audit séparé
- Cybersécurité complète (validation, RBAC, rate limiting, erreurs génériques)
- 4/7 bonus (dashboard web, JSON vs Protobuf, analyse URLs, RPC séparé)

7.2 Évolutions possibles

- Analyse des pièces jointes (détection .exe, .js, PDF avec JavaScript)
- Détection d'attaques homograph (Unicode Cyrillic/Latin)
- SPF/DKIM/DMARC via parsing des headers
- Circuit breaker pour les appels gRPC
- File d'attente asynchrone (Celery/RabbitMQ)
- Détection de langue pour multilinguisme

7.3 Apports pédagogiques

Ce projet a permis de mettre en pratique : - Les architectures distribuées et la communication inter-services (REST + gRPC) - La cybersécurité by design (authentification, autorisation, validation, audit) - Le déploiement de modèles ML en production - La gestion de projet en Methodologie Git (commits atomiques, branches, push régulier)

PhishGuard — Développé par Hatim Sami, Juin 2026